TypeScript — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Setup

02 Basic Types

03 Functions and Parameters

04 Interfaces and Type Aliases

05 Union, Intersection and Literal Types

06 Classes and Access Modifiers

Intermediate (6 topics)

07 Type Narrowing and Guards

08 Generics

09 Enums and Const Assertions

10 Modules and Namespaces

11 tsconfig and Strict Mode

12 Working with JavaScript Libraries

Advanced (6 topics)

13 Utility Types

14 Mapped and Conditional Types

15 Template Literal and Index Types

16 Decorators

17 Discriminated Unions and Exhaustiveness

18 Async Types and Error Handling

Master (4 topics)

19	Advanced Type Patterns
20	TypeScript with React, Node and Angular
21	Testing and Linting
22	Large Codebases: Monorepos and Project References

Beginner Level

01. Introduction and Setup

DEFINITION

TypeScript adds static types to JavaScript and compiles to plain JS. Install with `npm i -D typescript`, create `tsconfig.json` and compile with `tsc`.

SYNTAX

```
tsc --init
```

EXAMPLE

```
npm i -D typescript
npx tsc --init
npx tsc
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Like a spell-checker for a letter: TypeScript reads your code before you send it and underlines mistakes.

தமிழ்

கடிதம் அனுப்பும் முன் எழுத்துப் பிழை சோதிப்பான் போல TypeScript குறியீடு ஓடும் முன் தவறுகளை அடிக்கோடிடுகிறது.

PROS

- Bugs caught early
- Better editor help
- Safer refactoring

CONS

- Extra build step
- Learning curve

ADVANCED / NEW FEATURES

- Native Go-based compiler (TypeScript 7) in preview, much faster
- Node can strip types and run `.ts`

02. Basic Types

DEFINITION

`string`, `number`, `boolean`, arrays, tuples, `enum`, `any`, `unknown`, `void`, `never`. Type inference means you often need no annotation.

SYNTAX

```
let x: type = value;
```

EXAMPLE

```
let id: number = 1;
const tags: string[] = ['a'];
let pair: [string, number] = ['x', 1];
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Labels on grocery jars: 'sugar', 'salt'. You cannot put sugar in a jar marked salt without noticing.

தமிழ்

மளிகை ஜாடிகளில் 'சர்க்கரை', 'உப்பு' எனப் பெயரிட்டிருப்பது போல; உப்பு ஜாடியில் சர்க்கரை போட்டால் உடனே தெரிந்துவிடும்.

PROS

- Clear data contracts
- Inference reduces typing

CONS

- `any` removes safety
- Types can get complex

ADVANCED / NEW FEATURES

- `satisfies`
- `using` declarations (5.2)

03. Functions and Parameters

DEFINITION

Type parameters and return values; optional (`?`), default and rest parameters; function types.

SYNTAX

```
function f(a: number, b?: string): void { }
```

EXAMPLE

```
function greet(name: string, greeting = 'Hello'): string {
  return `${greeting}, ${name}`;
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A ticket counter form with clearly marked mandatory and optional boxes; function signatures state what is required.

தமிழ்

டிக்கெட் கவுண்டர் படிவத்தில் கட்டாய, விருப்ப கட்டங்கள் குறிக்கப்பட்டிருப்பது போல function signature தேவையானவற்றைத் தெளிவாகக் காட்டும்.

PROS

- Self-documenting APIs
- Safer calls

CONS

- Overloads can get verbose

ADVANCED / NEW FEATURES

- `NoInfer` utility (5.4)
- Inferred type predicates (5.5)

04. Interfaces and Type Aliases

DEFINITION

Describe object shapes with `interface` or `type`. Use `readonly` and optional properties.

SYNTAX

```
interface User { id: number; name?: string }
type ID = string | number;
```

EXAMPLE

```
interface User { readonly id: number; name: string; email?: string }
type ID = number | string;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

An Aadhaar card format: fixed fields such as name, DOB, number. An interface defines the exact shape of an object.

தமிழ்

ஆதார் அட்டை வடிவம்: பெயர், பிறந்த தேதி, எண் போன்ற நிலையான புலங்கள். Interface ஒரு object-இன் வடிவத்தை வரையறுக்கிறது.

PROS

- Clear contracts
- Declaration merging for interfaces

CONS

- Types disappear at runtime
- Interface vs type choice confuses

ADVANCED / NEW FEATURES

- Better `readonly` and `const` support

05. Union, Intersection and Literal Types

DEFINITION

`A | B` is either; `A & B` is both. Literal types restrict values to exact strings/numbers.

SYNTAX

```
type A = B | C;
type D = B & C;
```

EXAMPLE

```
type Status = 'idle' | 'loading' | 'error';
type Admin = User & { role: 'admin' };
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A railway ticket is either 'sleeper' or 'AC' (union); a student can be both 'sportsman' and 'topper' (intersection).

தமிழ்

ரயில் டிக்கெட் 'ஸ்லீப்பர்' அல்லது 'ஏசி' (union); ஒரு மாணவர் 'விளையாட்டு வீரர்' மற்றும் 'முதல் மாணவர்' (intersection) இருவரும் ஆகலாம்.

PROS

- Expressive modelling
- Exhaustive checking

CONS

- Large unions can slow the compiler

ADVANCED / NEW FEATURES

- Template literal types
- `const` type parameters

06. Classes and Access Modifiers

DEFINITION

Classes with `public`, `private`, `protected`, `readonly`, parameter properties, `abstract` classes and `implements`.

SYNTAX

```
class A { constructor(private x: number) {} }
```

EXAMPLE

```
class Account {
  constructor(private balance: number) {}
  deposit(n: number) { this.balance += n; }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bank locker: some compartments only the manager (private) can open, others customers (public).

தமிழ்

வங்கி லாக்கரில் சில அறைகளை மேலாளர் மட்டுமே (private) திறக்கலாம், சிலவற்றை வாடிக்கையாளர்கள் (public) திறக்கலாம்.

PROS

- Compile-time encapsulation
- Parameter properties reduce boilerplate

CONS

- `private` is erased at runtime (use `#private`)

ADVANCED / NEW FEATURES

- Standard ECMAScript decorators (5.0)

Intermediate Level

07. Type Narrowing and Guards

DEFINITION

Narrow a union using `typeof`, `instanceof`, `in`, equality and custom type guards (`x is T`).

SYNTAX

```
if (typeof x === 'string') { }
function isA(x): x is A { }
```

EXAMPLE

```
function len(x: string | string[]) {
  return typeof x === 'string' ? x.length : x.length;
}
const isUser = (o: any): o is User => 'id' in o;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A gatekeeper asks 'are you a student or a teacher?' before allowing different doors. Narrowing checks the kind of value before using it.

தமிழ்

வாயிலில் 'மாணவரா, ஆசிரியரா?' எனக் கேட்டு வெவ்வேறு கதவுகளுக்கு அனுப்புவது போல narrowing மதிப்பின் வகையைச் சரிபார்த்த பின்பே பயன்படுத்தும்.

PROS

- Safe use of unions
- No casts needed

CONS

- Custom guards can lie
- Complex flows may not narrow

ADVANCED / NEW FEATURES

- Inferred type predicates (5.5)
- Better narrowing in closures

08. Generics

DEFINITION

Write reusable code that works with many types while staying type-safe; constrain with `extends`.

SYNTAX

```
function f<T>(x: T): T { return x; }
```

EXAMPLE

```
function first<T>(items: T[]): T | undefined { return items[0]; }
function pick<T, K extends keyof T>(o: T, k: K): T[K] { return o[k]; }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A universal steel tiffin box: the box design is same, but it can carry idli, pongal or sambar. Generics reuse code for many types.

தமிழ்

ஒரே வடிவ டிபன் பாக்ஸ் இட்லி, பொங்கல், சாம்பார் எதையும் எடுத்துச் செல்லும்; generics பல வகைகளுக்கு ஒரே குறியீட்டைப் பயன்படுத்தும்.

PROS

- Reusable and type-safe
- Less duplication

CONS

- Hard error messages
- Over-abstraction

ADVANCED / NEW FEATURES

- `const` type parameters
- `NoInfer`

09. Enums and Const Assertions

DEFINITION

Prefer string unions or `as const` objects over `enum` in many cases.

EXAMPLE

```
const Roles = { Admin: 'admin', User: 'user' } as const;
type Role = typeof Roles[keyof typeof Roles];
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A fixed menu card of allowed choices; an `as const` object lists the exact options.

தமிழ்

அனுமதிக்கப்பட்ட தேர்வுகள் மட்டும் உள்ள நிலையான மெனு அட்டை போல `as const` object சரியான தேர்வுகளை வரையறுக்கும்.

PROS

- Exact literal types
- No runtime enum cost

CONS

- `enum` has quirks
- More boilerplate than enums

ADVANCED / NEW FEATURES

- `--erasableSyntaxOnly` flag discourages enums (5.8)

10. Modules and Namespaces

DEFINITION

Use ES `import/export`; `import type` imports types only. Configure `module` and `moduleResolution` in tsconfig.

SYNTAX

```
import type { A } from './a';
export { b };
```

EXAMPLE

```
import type { User } from './types';
export const x = 1;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A school has separate staff rooms per department; modules separate code but allow sharing through export/import.

தமிழ்

பள்ளியில் துறைவாரியாக ஆசிரியர் அறைகள் இருப்பது போல modules குறியீட்டைப் பிரித்து export/import மூலம் பகிர்கின்றன.

PROS

- Code organisation
- Tree shaking

CONS

- Module resolution config is confusing

ADVANCED / NEW FEATURES

- `verbatimModuleSyntax`
- `moduleResolution: bundler/nodenext`

11. tsconfig and Strict Mode

DEFINITION

Enable `"strict": true`. Key options: `target`, `lib`, `noImplicitAny`, `strictNullChecks`, `paths`, `outDir`.

SYNTAX

```
{ "compilerOptions": { "strict": true } }
```

EXAMPLE

```
{ "compilerOptions": { "strict": true, "target": "ES2022", "module": "ESNext" } }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A school rulebook: the stricter the rules, the fewer problems later. `strict` mode enables all safety checks.

தமிழ்

பள்ளி விதிமுறைகள் கடுமையாக இருந்தால் பின்னர் பிரச்சனை குறையும்; `strict` அனைத்துப் பாதுகாப்புச் சோதனைகளையும் இயக்குகிறது.

PROS

- Maximum safety
- Better inference

CONS

- Migrating legacy code is hard

ADVANCED / NEW FEATURES

- `--strict` now bundles newer checks
- `--erasableSyntaxOnly`

12. Working with JavaScript Libraries

DEFINITION

Install types with `@types/package` or write `.d.ts` declaration files for untyped libraries.

SYNTAX

```
npm i -D @types/express
```

EXAMPLE

```
npm i -D @types/node @types/express
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Buying a Tamil-English dictionary so you can understand a foreign book; `@types` packages describe JavaScript libraries.

தமிழ்

வெளிநாட்டு நூலைப் புரிந்துகொள்ள தமிழ்-ஆங்கில அகராதி வாங்குவது போல `@types` தொகுப்புகள் JavaScript நூலகங்களை விவரிக்கின்றன.

PROS

- Autocomplete for JS libraries
- Community maintained

CONS

- Types may lag behind libraries

ADVANCED / NEW FEATURES

- More packages ship their own types

Advanced Level

13. Utility Types

DEFINITION

Built-ins: `Partial`, `Required`, `Readonly`, `Pick`, `Omit`, `Record`, `ReturnType`, `Parameters`, `Awaited`, `NonNullable`.

SYNTAX

```
Partial<T>, Pick<T, K>, Omit<T, K>, Record<K, V>
```

EXAMPLE

```
type UserPreview = Pick<User, 'id' | 'name'>;
type Draft = Partial<User>;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A form template where you can mark some fields optional or pick only a few fields; utility types reshape existing types.

தமிழ்

படிவ மாதிரியில் சில புலங்களை விருப்பமாக்கலாம் அல்லது சிலவற்றை மட்டும் தேர்ந்தெடுக்கலாம்; utility types ஏற்கனவே உள்ள type-ஐ மாற்றியமைக்கின்றன.

PROS

- Avoid duplication
- Always in sync with source types

CONS

- Nested utility types hurt readability

ADVANCED / NEW FEATURES

- `NoInfer`, `Awaited`
- `Disposable` types

14. Mapped and Conditional Types

DEFINITION

Transform types programmatically with `{ [K in keyof T]: ... }` and `T extends U ? X : Y`.

SYNTAX

```
type M<T> = { [K in keyof T]: T[K] | null };
```

EXAMPLE

```
type Nullable<T> = { [K in keyof T]: T[K] | null };
type ElementOf<T> = T extends (infer E)[] ? E : never;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A photocopier that stamps every page the same way; mapped types transform every property of a type.

தமிழ்

ஒவ்வொரு பக்கத்திலும் ஒரே முத்திரையிடும் நகலெடுப்பான் போல mapped types ஒரு type-இன் ஒவ்வொரு புலத்தையும் மாற்றும்.

PROS

- Powerful type transforms
- DRY

CONS

- Hard to read
- Slow compile for huge types

ADVANCED / NEW FEATURES

- Key remapping with `as`
- Variadic tuple types

15. Template Literal and Index Types

DEFINITION

Build string types: `` `${A}-${B}` ``; use `keyof`, `typeof` and `indexed access` `T['key']`.

SYNTAX

```
type E = `on${Capitalize<Name>}`;
```

EXAMPLE

```
type Event = `on${Capitalize<'click' | 'hover'>}`; // 'onClick' | 'onHover'
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Auto-generating shop bill numbers like INV-2026-001; template literal types build string types.

தமிழ்

INV-2026-001 போன்ற பில் எண்களை தானாக உருவாக்குவது போல template literal types string வகைகளை உருவாக்கும்.

PROS

- Precise string typing
- Great for routes and events

CONS

- Complex types are hard to debug

ADVANCED / NEW FEATURES

- Intrinsic helpers like `Uppercase`, `Capitalize`

16. Decorators

DEFINITION

Decorators add metadata or behaviour to classes and members; used heavily by Angular and NestJS.

SYNTAX

```
@decorator
class A { }
```

EXAMPLE

```
function Log(target: any, key: string, d: PropertyDescriptor) {
  const f = d.value;
  d.value = (...a: any[]) => { console.log(key, a); return f(...a); };
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A name badge added to any employee without changing the person; decorators attach extra behaviour to classes.

தமிழ்

எந்த ஊழியரையும் மாற்றாமல் பெயர் அட்டையை இணைப்பது போல decorators வகுப்புகளுக்கு கூடுதல் செயல்பாட்டை இணைக்கின்றன.

PROS

- Clean cross-cutting features
- Used by Angular/NestJS

CONS

- Older and new decorator forms differ
- Metadata needs extra libs

ADVANCED / NEW FEATURES

- Standard (TC39) decorators supported since TS 5.0

17. Discriminated Unions and Exhaustiveness

DEFINITION

Add a literal `kind` tag to each union member, then `switch` safely; use `never` to force handling all cases.

SYNTAX

```
type S = { kind: 'a' } | { kind: 'b' };
```

EXAMPLE

```
type Shape = { kind: 'circle'; r: number } | { kind: 'square'; s: number };
function area(s: Shape) { switch (s.kind) { case 'circle': return Math.PI * s.r ** 2; case
'square': return s.s ** 2; } }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Different coloured ration cards (white, pink, green) each get different benefits; a `kind` tag tells which card you hold.

தமிழ்

வெவ்வேறு நிற ரேஷன் அட்டைகளுக்கு வெவ்வேறு சலுகைகள் கிடைப்பது போல `kind` குறியீடு எந்த வகை என்று சொல்கிறது.

PROS

- Compiler forces handling all cases
- Clear state machines

CONS

- Verbose for many variants

ADVANCED / NEW FEATURES

- Better exhaustiveness checks with `satisfies never`

18. Async Types and Error Handling

DEFINITION

Type promises as `Promise<T>`; model results with union types (`Result<T,E>`) rather than throwing everywhere.

SYNTAX

```
Promise<T>
```

```ts
type Result<T> = { ok: true; value: T } | { ok: false; error: string };
```

EXAMPLE

```ts
type Result<T> = { ok: true; value: T } | { ok: false; error: string };
async function load(): Promise<Result<User>> { /* ... */ }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A courier returns either the parcel or a failure slip; typing results makes both cases explicit.

தமிழ்

கூரியர் பார்சலையோ தோல்விச் சீட்டையோ திருப்பித் தரும்; result types இரண்டு நிலைகளையும் தெளிவாக்குகின்றன.

PROS

- Explicit error handling
- Safer async code

CONS

- More boilerplate than throwing

ADVANCED / NEW FEATURES

- `Awaited<T>`
- Better async inference

Master Level

19. Advanced Type Patterns

DEFINITION

Branded types, builder patterns, recursive types, `satisfies` and variadic tuples for very safe APIs.

SYNTAX

```
type Id = string & { __brand: 'Id' };
```

EXAMPLE

```
type UserId = string & { __brand: 'UserId' };
const cfg = { port: 3000 } satisfies Record<string, number>;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Two keys look similar but belong to different locks; branded types stop you using the wrong key.

தமிழ்

பார்க்க ஒரே மாதிரி இரு சாவிகள் வெவ்வேறு பூட்டுகளுக்கானவை; branded types தவறான சாவியைப் பயன்படுத்தாமல் தடுக்கின்றன.

PROS

- Prevents mixing similar values
- Zero runtime cost

CONS

- Needs helper constructors

ADVANCED / NEW FEATURES

- `satisfies` operator
- `NoInfer`

20. TypeScript with React, Node and Angular

DEFINITION

React: type props and hooks; Node: `@types/node` + `tsx` / `ts-node`; Angular: TypeScript is built in with strict templates.

EXAMPLE

```
type Props = { title: string };
```

```
const Card = ({ title }: Props) => <h1>{title}</h1>;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A translator who speaks both Tamil and English; TypeScript sits between your framework and plain JavaScript.

தமிழ்

தமிழ், ஆங்கிலம் இரண்டும் பேசும் மொழிபெயர்ப்பாளர் போல TypeScript உங்கள் framework-க்கும் JavaScript-க்கும் இடையில் உள்ளது.

PROS

- Type-safe props and APIs
- Great IDE experience

CONS

- Framework type quirks
- Build config complexity

ADVANCED / NEW FEATURES

- Angular strict templates
- React `use()` /Server Components typing
- `tsx` and `tsc --noEmit` in CI

21. Testing and Linting

DEFINITION

Test with Vitest/Jest + `ts-jest`; lint with `typescript-eslint`; use `tsc --noEmit` in CI.

SYNTAX

```
npx tsc --noEmit
```

EXAMPLE

```
npx tsc --noEmit
npx eslint .
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A school inspector checks everything before the annual day; linting and `tsc` inspect your code before release.

தமிழ்

ஆண்டு விழாவுக்கு முன் ஆய்வாளர் அனைத்தையும் சோதிப்பது போல lint, `tsc` வெளியீட்டுக்கு முன் குறியீட்டைச் சோதிக்கும்.

PROS

- Quality gate in CI
- Consistent style

CONS

- Slow on huge repos
- Rule configuration effort

ADVANCED / NEW FEATURES

- typescript-eslint flat config
- Native compiler speed-ups

22. Large Codebases: Monorepos and Project References

DEFINITION

Use workspaces, `paths` aliases and `composite` project references for fast incremental builds.

SYNTAX

```
{ "references": [{ "path": "../core" }] }
```

EXAMPLE

```
{ "references": [{ "path": "../shared" }] }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A big company with separate departments building their own parts and joining at the end; project references build large code bases in pieces.

தமிழ்

பெரிய நிறுவனத்தில் துறைகள் தங்கள் பகுதிகளைத் தனித்தனியாகக் கட்டி இறுதியில் இணைப்பது போல project references பெரிய குறியீட்டைத் துண்டுகளாகக் கட்டுகின்றன.

PROS

- Faster incremental builds
- Clear boundaries

CONS

- Setup complexity

ADVANCED / NEW FEATURES

- Isolated declarations
- Faster native builds (TS 7)

Interview Question Bank

Q1. TypeScript vs JavaScript?
TypeScript is a typed superset that compiles to JavaScript, catching errors at compile time.

Q2. interface vs type?
Interfaces can be merged and extended; types can also express unions, tuples and mapped types.

Q3. any vs unknown vs never?
`any` disables checks, `unknown` requires narrowing, `never` is a value that cannot occur.

Q4. What are generics?
Type parameters that make functions and classes reusable and type-safe.

Q5. What is `strictNullChecks`?
Makes `null` and `undefined` separate types so you must handle them.

Q6. What are utility types?
Built-in generic helpers such as `Partial`, `Pick`, `Omit`, `Record`.

Q7. What is type narrowing?
Refining a broad type to a specific one with checks like `typeof` or `in`.

Q8. What is a discriminated union?
A union whose members share a literal tag property that enables safe `switch` narrowing.

Q9. Enum vs union of literals?
Unions emit no runtime code and are simpler; enums create an object at runtime.

Q10. What does `as const` do?
Makes values readonly and narrows them to literal types.

Capstone Project Ideas

1. Typed CLI to-do tool
2. Typed REST API client library
3. Generic data table component
4. State machine with discriminated unions
5. Type-safe event emitter